

8 Essential Node.js Interview Questions *

Toptal sourced essential questions that the best Node.js developers and engineers can answer. Driven from our community, we encourage experts to submit questions and offer feedback.

[Hire a Top Node.js Developer Now](#)

 **Toptal** is an exclusive network of the top freelance software developers, designers, marketing experts, product managers, project managers, and finance experts in the world. Top companies hire Toptal freelancers for their most important projects.

INTERVIEW QUESTIONS

✓ 1. How does Node.js handle child threads?

[Hide answer](#)



Node.js, in its essence, is a **single thread** process. It does not expose child threads and thread management methods to the developer. Technically, Node.js *does* spawn child threads for certain tasks such as asynchronous I/O, but these run behind the scenes and do not execute any application JavaScript code, nor block the main event loop.

If threading support is desired in a Node.js application, there are tools available to enable it, such as the [ChildProcess](#) module. In fact, [Node.js 12 has experimental support for threads](#).

✓ 2. How does Node.js support multi-processor platforms, and does it fully utilize all processor resources?

[Hide answer](#)



Since Node.js is by default a **single thread** application, it will run on a single processor core and will not take full advantage of multiple core resources. However, Node.js provides support for deployment on multiple-core systems, to take greater advantage of the hardware. The [Cluster](#) module is one of the core Node.js modules and it allows running multiple Node.js worker processes that will share the same port.

✓ **3. What is typically the first argument passed to a Node.js callback handler?**

[Hide answer](#)



Node.js core modules, as well as most of the community-published ones, follow a pattern whereby the first argument to any callback handler is an optional error object. If there is no error, the argument will be null or undefined.

A typical callback handler could therefore perform error handling as follows:

```
function callback(err, results) {  
  // usually we'll check for the error before handling results  
  if(err) {  
    // handle error somehow and return  
  }  
  // no error, perform standard callback handling  
}
```

Apply to Join Toptal's Development Network
and enjoy reliable, steady, remote [Freelance Node.js Developer Jobs](#)

[Apply as a Freelancer](#)

✓ **4. What is REPL? What purpose it is used for?**

[Hide answer](#)



REPL stands for (READ, EVAL, PRINT, LOOP). Node.js comes with a bundled REPL environment. This allows for the easy creation of CLI (Command Line Interface) applications.

✓ 5. Consider the following JavaScript code:

```
console.log("first");
setTimeout(() => {
  console.log("second");
}, 0);
console.log("third");
```

The output will be:

```
first
third
second
```

Assuming that this is the desired behavior, how else might we write this code?

[Hide answer](#)



Way back when, Node.js version 0.10 introduced `setImmediate`, which is equivalent to `setTimeout(fn, 0)`, but with some slight advantages.

`setTimeout(fn, delay)` calls the given callback `fn` after the given `delay` has elapsed (in milliseconds). However, the callback is not executed immediately at this time, but added to the function queue so that it is executed **as soon as possible**, after all the currently executing and currently queued event handlers have completed. Setting the delay to 0 adds the callback to the queue immediately so that it is executed as soon as all currently-queued functions are finished.

`setImmediate(fn)` achieves the same effect, except that it doesn't use the queue of functions. Instead, it checks the queue of I/O event handlers. If all I/O events in the current snapshot are processed, it executes the callback. It queues them immediately after the last I/O handler, somewhat like `process.nextTick`. This is faster than `setTimeout(fn, 0)`.

So, the above code can be written in Node.js as:

```
console.log("first");
setImmediate(() => {
  console.log("second");
});
console.log("third");
```

✓ 6. What is the preferred method of resolving unhandled exceptions in Node.js?

[Hide answer](#)



Unhandled exceptions in Node.js can be caught at the `Process` level by attaching a handler for `uncaughtException` event.

```
process.on('uncaughtException', (err) => {  
  console.log(`Caught exception: ${err}`);  
});
```

However, `uncaughtException` is a very crude mechanism for exception handling and may be removed from Node.js in the future. An exception that has bubbled all the way up to the `Process` level means that your application, and Node.js may be in an undefined state, and the only sensible approach would be to restart everything.

The preferred way is to add another layer between your application and the Node.js process which is called the [domain](#).

Domains provide a way to handle multiple different I/O operations as a single group. So, by having your application, or part of it, running in a separate domain, you can safely handle exceptions at the domain level, before they reach the `Process` level.

However, domains have been [pending deprecation](#) for a few years—since Node.js 4. It's *possible* a more future-proof approach would be to use [zones](#).

✓ 7. Consider following code snippet:

```
{  
  console.time("loop");  
  for (var i = 0; i < 1000000; i += 1){  
    // Do nothing  
  }  
  console.timeEnd("loop");  
}
```

The time required to run this code in Google Chrome is considerably more than the time required to run it in [Node.js](#). Explain why this is so, even though both use the v8 JavaScript Engine.

[Hide answer](#)



Within a web browser such as Chrome, declaring the variable `i` outside of any function's scope makes it global and therefore binds it as a property of the `window` object. As a result, running this code in a web browser requires repeatedly resolving the property `i` within the heavily populated `window` namespace in each iteration of the `for` loop.

In Node.js, however, declaring any variable outside of any function's scope binds it only to the module's own scope (not the `window` object) which therefore makes it much easier and faster to resolve.

It's also worth noting that using `let` instead of `var` in the `for` loop declaration can reduce the loop's run time by over 50%. But such a change assumes you know [the difference between `let` and `var`](#) and whether this will have an effect on the behavior of your specific loop.

✓ 8. What is “callback hell” and how can it be avoided?

[Hide answer](#)



“Callback hell” refers to heavily nested callbacks that have become unweildy or unreadable.

An example of heavily nested code is below:

```
query("SELECT clientId FROM clients WHERE clientName='picanteverde';", function(id){
  query(`SELECT * FROM transactions WHERE clientId=${id}`, function(transactions){
    transactions.each((transac) => {
      query(`UPDATE transactions SET value = ${transac.value*0.1} WHERE id=${transac.id}`);
      if(!error){
        console.log("success!!");
      }else{
        console.log("error");
      }
    });
  });
});
```

At one point, the primary method to fix callback hell was **modularization**. The callbacks are broken out into independent functions which can be called with some parameters. So the first level of improvement might be:

```
const logError = (error) => {
  if(!error){
    console.log("success!!");
  }else{
    console.log("error");
  }
},
```

```
updateTransaction = (t) => {
  query(`UPDATE transactions SET value = ${t.value*0.1} WHERE id=${t.id}, logError);
},
handleTransactions = (transactions) => {
  transactions.each(updateTransaction);
},
handleClient = (id) => {
  query(`SELECT * FROM transactions WHERE clientId=${id}`, handleTransactions);
};

query("SELECT clientId FROM clients WHERE clientName='picanteverde';",handleClient);
```

Even though this code is much easier to read, and we created some functions that we can even reuse later, in some cases it may be appropriate to use a more robust solution in the form of **promises**. Promises allow additional desirable behavior such as error propagation and chaining. Node.js includes native support for them.

Additionally, a more supercharged solution to callback hell was provided by **generators**, as these can resolve execution dependency between different callbacks. However, generators are much more advanced and it might be overkill to use them for this purpose. To read more about generators you can start with [this post](#).

However, these approaches are pretty dated at this point. The current solution is to [use `async/await`](#)—an approach that leverages Promises and finally makes it easy to flatten the so-

- * These sample questions are intended as a starting point for your interview process. If you need additional help, explore our [hiring resources](#)—or let Toptal find the best [developers](#), [designers](#), [marketing experts](#), [product managers](#), [project managers](#), and [finance experts](#) for you.

WHY TOPTAL

Tired of interviewing candidates? Let Toptal help you find the right fit.

Get the world's top talent, on demand.

[Hire a Top Node.js Developer Now](#)

OUR EXCLUSIVE NETWORK OF NODE.JS DEVELOPERS



Yashwanth Reddy

✓ Verified Expert in Engineering

Freelance Node.js Developer from United States
Member since July 16, 2024

Looking to land a job as a Node.js Developer?

Let Toptal find the right job for you.

[Apply as a Node.js Developer](#)

JOB OPPORTUNITIES FROM OUR NETWORK

Remote Node.js Developer Job for Travel Marketplace (Part-time)

1 day ago · Remote · Part Time



Submit an interview question

Submitted questions and answers are subject to review and editing, and may or may not be selected for posting, at the sole discretion of Toptal, LLC.

Name *

Email *

Enter Your Question Here ... *

Enter Your Answer Here ... *

☐ * I agree with the Terms and Conditions of Toptal, LLC's [Privacy Policy](#)

* All fields are required

Submit a Question

Looking for Node.js Developers?

Looking for [Node.js Developers](#)? Check out Toptal's Node.js developers.



Yashwanth Reddy

Freelance Node.js Developer

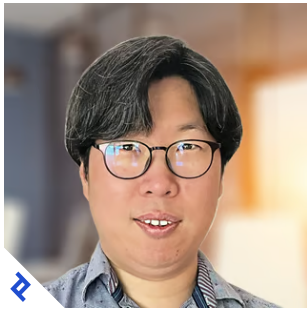
United States Toptal Member Since July 16, 2024

Yashwanth is a full-stack web developer and DevOps engineer with 11+ years in IT. He has additional experience in web application development using HTML5, CSS3, JavaScript, React, Node.js, Docker, Kubernetes,...

[Show More](#)

View Yashwanth

- Node.js
- Linux
- JavaScript
- Docker
- Kubernetes
- AWS IoT
- Terraform
- React.js
- Python
- Next.js
- FastAPI



Woonchan Cho

Freelance Node.js Developer

United States Toptal Member Since May 16, 2022

Woonchan is a self-motivated full-stack developer and data analyst with over 10 years of experience managing, developing, and maintaining various systems using React, Java, Spring, and Node.js. He is swift in adapting to...

[Show More](#)

[View Woonchan](#)

Node.js

Web Development

Java

Software Development

JavaScript

TypeScript

React.js

Full-stack

UI Development



Mike Timashov

Freelance Node.js Developer

Poland Toptal Member Since March 23, 2021

Mike is an expert software engineer with more than ten years of professional experience working on enterprise and startup projects. He's passionate about learning, building world-class quality software, and...

[Show More](#)

[View Mike](#)

Node.js

Express.js

TypeScript

Software Development

iOS

Android

MySQL

JavaScript

Firebase

REST API

Toptal Connects the Top 3% of Freelance
Talent All Over The World.

Join the Toptal community.

[Learn more](#)

Skills in High Demand by Clients

[JavaScript Developer Jobs](#)

[React.js Developer Jobs](#)

[Python Developer Jobs](#)

[WordPress Developer Jobs](#)

[Node.js Developer Jobs](#)

[AWS Developer Jobs](#)

About

[Top 3%](#)

[Clients](#)

[Toptal Talent Assessment](#)

[Ultimate Freelancing Guide](#)

[Blog](#)

[Community](#)

[About Us](#)

Contact

[Contact Us](#)

[Press Center](#)

[Careers](#)

[FAQ](#)



The World's Top Talent, On Demand™



Copyright 2010 - 2024 Toptal, LLC

[Privacy Policy](#) [Website Terms](#)